

Listes déroulantes (select / option) avec Thymeleaf

1. Principes

Une liste déroulante HTML est composée d'un élément `<select>` contenant des éléments `<option>`. Avec Thymeleaf et Spring MVC, on lie le `<select>` à une propriété d'un objet formulaire (*form-backing object*) et on génère les `<option>` à partir d'une collection de données du modèle.

Trois attributs sont mis en jeu :

Attribut	Où	Rôle
th:object	<code><form></code>	Associe le formulaire à un objet du modèle
th:field="*{countryId}"	<code><select></code>	Lie la liste à la propriété <code>countryId</code> : génère id et name, et sélectionne automatiquement l'option correspondant à la valeur de la propriété
th:value	<code><option></code>	Valeur envoyée au serveur pour cette option

Points à retenir :

- Pour que la présélection automatique fonctionne, chaque `<option>` doit avoir un `th:value`. Thymeleaf compare cette valeur à celle de la propriété liée et ajoute l'attribut `selected` à l'option correspondante. Cette comparaison se fait via le service de conversion de Spring : une propriété Long et une valeur d'option textuelle "2" sont reconnues comme égales.
- L'option vide `<option value="">-- Choisir --</option>` permet de ne rien sélectionner. Spring convertit la chaîne vide en null pour une propriété de type Long : on peut donc détecter l'absence de choix avec `@NotNull`.
- Après un échec de validation, `th:field` **re-sélectionne** automatiquement l'option que l'utilisateur avait choisie.

Dans ce document, nous construisons la liste de trois façons : à partir d'une List d'objets (exemple 1), à partir d'une Map parcourue par entrées (exemple 2) et à partir d'une Map parcourue par clés (exemple 3).

2. Les classes Java

Les trois classes suivantes sont communes à tous les exemples.

`Country.java`

```
package local.epul4a.demosbt.model;
```

```
public class Country {
```

```
private final Long countryId;
private final String countryCode;
private final String countryName;

public Country(Long countryId, String countryCode, String countryName) {
    this.countryId = countryId;
    this.countryCode = countryCode;
    this.countryName = countryName;
}

public Long getCountryId() { return countryId; }
public String getCountryCode() { return countryCode; }
public String getCountryName() { return countryName; }
}
```

PersonForm.java

L'objet formulaire porte les données saisies par l'utilisateur. On y déclare les règles de validation (Bean Validation, dépendance *Validation* du projet) :

```
package local.epul4a.demosbt.form;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

public class PersonForm {

    @NotBlank(message = "Le nom complet est obligatoire")
    private String fullName;

    @NotNull(message = "Veuillez choisir un pays")
    private Long countryId;

    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }

    public Long getCountryId() { return countryId; }
    public void setCountryId(Long countryId) { this.countryId = countryId; }
}
```

CountryDAO.java

Ce DAO simule un accès aux données avec une liste en mémoire. Dans une vraie application, il serait remplacé par un JpaRepository (Spring Data JPA).

```
package local.epul4a.demosbt.dao;

import java.util.LinkedHashMap;
import java.util.List;
```

```
import java.util.Map;
import java.util.Optional;

import local.epul4a.demosbt.model.Country;
import org.springframework.stereotype.Repository;

@Repository
public class CountryDAO {

    private static final List<Country> COUNTRIES = List.of(
        new Country(1L, "AL", "Albanie"),
        new Country(2L, "FR", "France"),
        new Country(3L, "GB", "Royaume-Uni"));

    public List<Country> getCountries() {
        return COUNTRIES;
    }

    // LinkedHashMap : conserve l'ordre d'insertion (un HashMap n'a pas
    // d'ordre garanti)
    public Map<Long, String> getMapCountries() {
        Map<Long, String> map = new LinkedHashMap<>();
        for (Country c : COUNTRIES) {
            map.put(c.getCountryId(), c.getCountryName());
        }
        return map;
    }

    public Optional<Country> findById(Long id) {
        return COUNTRIES.stream()
            .filter(c -> c.getCountryId().equals(id))
            .findFirst();
    }
}
```

3. Le contrôleur

Le contrôleur affiche le formulaire (requête GET) et traite sa soumission (requête POST). Les trois exemples ne diffèrent que par leur template : on les regroupe donc dans un seul contrôleur, avec une variable de chemin n limitée à 1, 2 ou 3 par une expression régulière.

```
package local.epul4a.demosbt.controller;

import java.util.List;
import java.util.Map;

import jakarta.validation.Valid;
import local.epul4a.demosbt.dao.CountryDAO;
```

```
import local.epul4a.demosbt.form.PersonForm;
import local.epul4a.demosbt.model.Country;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
public class SelectOptionController {

    private final CountryDAO countryDAO;

    public SelectOptionController(CountryDAO countryDAO) {
        this.countryDAO = countryDAO;
    }

    // Ces deux méthodes alimentent le modèle pour TOUS les handlers du
    // contrôleur,
    // y compris lorsqu'on réaffiche le formulaire après une erreur de
    // validation.
    @ModelAttribute("countries")
    public List<Country> countries() {
        return countryDAO.getCountries();
    }

    @ModelAttribute("mapCountries")
    public Map<Long, String> mapCountries() {
        return countryDAO.getMapCountries();
    }

    @GetMapping("/selectOptionExample{n:[1-3]}")
    public String showForm(@PathVariable("n") int n, Model model) {
        model.addAttribute("personForm", new PersonForm());
        return "selectOptionExample" + n;
    }

    @PostMapping("/selectOptionExample{n:[1-3]}")
    public String submit(@PathVariable("n") int n,
        @Valid @ModelAttribute("personForm") PersonForm
personForm,
        BindingResult result,
        RedirectAttributes redirect) {
        if (result.hasErrors()) {
            return "selectOptionExample" + n; // on réaffiche le formulaire
        }
        String countryName = countryDAO.findById(personForm.getCountryId())
            .map(Country::getCountryName)
            .orElse("inconnu");
    }
}
```

```
        redirect.addFlashAttribute("fullName", personForm.getFullName());
        redirect.addFlashAttribute("countryName", countryName);
        return "redirect:/selectOptionResult";    // Post/Redirect/Get
    }

    @GetMapping("/selectOptionResult")
    public String result() {
        return "selectOptionResult";
    }
}
```

Explications :

- **@ModelAttribute sur une méthode** : la liste des pays est ajoutée au modèle avant chaque handler. Sans cela, le réaffichage du formulaire après une erreur de validation aurait un <select> vide, car la liste n'aurait pas été rechargée. C'est une erreur très fréquente.
- **@Valid et BindingResult** : le BindingResult doit suivre immédiatement l'objet validé.
- **RedirectAttributes.addFlashAttribute** : les valeurs sont conservées pour la requête suivante (la redirection), puis supprimées. Elles évitent de passer des données dans l'URL.

4. Les templates

Le menu commun (fragment)

L'original incluait un menu avec `th:include`, attribut **déprécié** depuis Thymeleaf 3.0. On définit le menu comme un fragment et on l'insère avec `th:replace` et une expression de fragment `~{...}`.

templates/fragments/_menu.html :

```
<nav th:fragment="menu" xmlns:th="http://www.thymeleaf.org">
  <a th:href="@{/selectOptionExample1}">Exemple 1</a> |
  <a th:href="@{/selectOptionExample2}">Exemple 2</a> |
  <a th:href="@{/selectOptionExample3}">Exemple 3</a>
</nav>
```

Insertion dans une page :

```
<div th:replace="~{fragments/_menu :: menu}"></div>
```

Exemple 1 : <select> + List

selectOptionExample1.html :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
```

```
<head>
  <meta charset="UTF-8" />
  <title>Select Option - exemple 1</title>
</head>
<body>
  <div th:replace=~{fragments/_menu :: menu}></div>

  <h2>Select Option - exemple 1 (List)</h2>

  <form th:action="@{/selectOptionExemple1}" th:object="${personForm}"
method="post">

    <div>
      <label for="fullName">Nom complet :</label>
      <input type="text" th:field="*{fullName}" />
      <span th:if="${#fields.hasErrors('fullName')}" th:errors="*{fullName}"
style="color: red;">Erreur</span>
    </div>

    <div>
      <label for="countryId">Pays :</label>
      <select th:field="*{countryId}">
        <option value="">-- Choisir --</option>
        <option th:each="country : ${countries}"
th:value="${country.countryId}"
th:text="${country.countryName}">Pays</option>
      </select>
      <span th:if="${#fields.hasErrors('countryId')}"
th:errors="*{countryId}"
style="color: red;">Erreur</span>
    </div>

    <input type="submit" value="Valider" />
  </form>
</body>
</html>
```

Remarques :

- L'original oubliait `th:action` et n'avait aucun traitement du POST : le formulaire ne faisait rien. Ici, il envoie les données à `/selectOptionExemple1`.
- L'élément `<option>` s'écrit avec une balise fermante explicite : une balise auto-fermante `<option ... />` n'est pas valide en HTML5.
- On utilise `th:text` (qui échappe le HTML) à la place de `th:utext`.

Exemple 2 : <select> + Map parcourue par entrées

Lorsqu'on itère directement sur une Map, chaque élément est un Map.Entry : on accède à la clé avec .key et à la valeur avec .value (voir le document sur les boucles). L'appel `${mapCountries.entrySet()}` de l'original est donc inutile.

```
<select th:field="*{countryId}">
  <option value="">-- Choisir --</option>
  <option th:each="entry : ${mapCountries}"
    th:value="${entry.key}"
    th:text="${entry.value}">Pays</option>
</select>
```

Le reste de la page (menu, formulaire, champ fullName) est identique à l'exemple 1.

Exemple 3 : <select> + Map parcourue par clés

```
<select th:field="*{countryId}">
  <option value="">-- Choisir --</option>
  <option th:each="key : ${mapCountries.keySet()}"
    th:value="${key}"
    th:text="${mapCountries.get(key)}">Pays</option>
</select>
```

Cette variante fonctionne, mais elle effectue une recherche dans la Map pour chaque option : l'exemple 2 est plus direct et plus lisible.

La page de résultat

selectOptionResult.html :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Résultat</title>
</head>
<body>
  <div th:replace="~{fragments/_menu :: menu}"></div>

  <h2>Résultat</h2>
  <p th:if="${fullName}">
    <span th:text="${fullName}">Nom</span> habite en
    <b th:text="${countryName}">Pays</b>.
  </p>
  <p th:unless="${fullName}">Aucune donnée : remplissez d'abord un
formulaire.</p>
</body>
</html>
```

5. Variantes utiles

Sélection multiple

Pour permettre plusieurs choix, on lie le `<select multiple>` à une propriété de type collection :

```
// dans le formulaire
private List<Long> countryIds = new ArrayList<>(); // avec getter et setter

<select multiple th:field="*{countryIds}">
  <option th:each="country : ${countries}"
    th:value="${country.countryId}"
    th:text="${country.countryName}">Pays</option>
</select>
```

Thymeleaf sélectionne alors chaque option dont la valeur figure dans la liste.

Présélectionner une valeur : `th:selected`

Lorsque la liste n'est pas liée à un objet formulaire (pas de `th:field`), on contrôle la sélection à la main avec `th:selected` :

```
<select name="countryId">
  <option th:each="country : ${countries}"
    th:value="${country.countryId}"
    th:text="${country.countryName}"
    th:selected="${country.countryId ==
defaultCountryId}">Pays</option>
</select>
```

Ne combinez **pas** `th:selected` avec `th:field` : ce dernier gère déjà la sélection.

Présélectionner une valeur avec `th:field`

Il suffit d'initialiser la propriété de l'objet formulaire avant d'afficher la page :

```
PersonForm form = new PersonForm();
form.setCountryId(2L); // La France est présélectionnée
model.addAttribute("personForm", form);
```

Grouper les options : `<optgroup>`

```
<select th:field="*{countryId}">
  <optgroup th:each="entry : ${countriesByContinent}"
th:label="${entry.key}">
    <option th:each="c : ${entry.value}"
      th:value="${c.countryId}"
th:text="${c.countryName}">Pays</option>
  </optgroup>
</select>
```

Ici, `countriesByContinent` est une `Map<String, List<Country>>` placée dans le modèle.

6. Erreurs fréquentes

Symptôme	Cause probable
La liste est vide après une erreur de validation	La liste n'est plus dans le modèle au réaffichage : utiliser <code>@ModelAttribute</code> sur une méthode
Aucune option n'est présélectionnée	<code>th:value</code> manquant sur les <code><option></code> , ou valeurs de types incompatibles
Erreur « Neither BindingResult nor plain target object for bean name »	L'objet formulaire n'a pas été placé dans le modèle avant l'affichage
Le POST renvoie une erreur 405 ou 404	<code>th:action</code> absent, ou aucun handler <code>@PostMapping</code> pour cette URL
Les pays apparaissent dans un ordre aléatoire	Utilisation d'un <code>HashMap</code> : préférer <code>LinkedHashMap</code> ou <code>TreeMap</code>

Résumé

- `th:field` lie le `<select>` à une propriété du formulaire et sélectionne l'option correspondante ; chaque `<option>` doit avoir un `th:value`.
- On génère les options avec `th:each`, depuis une `List` (propriétés des objets) ou une `Map` (`.key`, `.value`).
- L'option vide combinée à `@NotNull` impose un choix ; les erreurs s'affichent avec `th:errors`.
- Les listes de choix doivent être rechargées au réaffichage du formulaire : `@ModelAttribute` sur une méthode.
- Un formulaire a besoin de `th:action` et d'un handler POST ; on redirige après succès (Post/Redirect/Get).
- `th:include` est déprécié : utiliser `th:replace` / `th:insert` avec `~{...}`.

Exercices proposés

1. Ajouter un champ « Ville » dont la liste dépend du pays choisi (indication : charger les villes par une seconde requête GET ou avec `htmx`).
2. Transformer l'exemple 1 en liste à choix multiples avec `List<Long> countryIds`, et afficher les pays sélectionnés dans la page de résultat.
3. Remplacer `CountryDAO` par un `JpaRepository<Country, Long>` avec une base H2, en annotant `Country` comme entité.
4. Présélectionner la France à l'ouverture du formulaire, puis expliquer pourquoi la présélection disparaît si l'on utilise `th:selected` en plus de `th:field`.

5. Écrire un test `@WebMvcTest` qui soumet le formulaire sans pays et vérifie que le modèle contient une erreur sur `countryId` et que la liste `countries` est toujours présente.